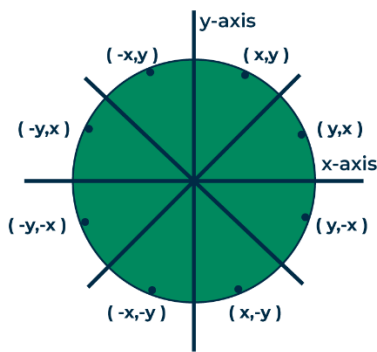


Unit 2. Line, Circle, and Polygon.

2.2 Circle generating algorithms:

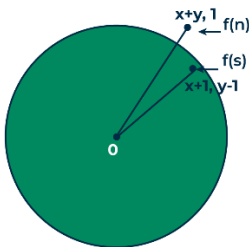
Bresenham's circle drawing algorithm

So, for whole 360 degree of circle we will divide it in 8-parts each octant of 45 degree. In order to do that we will use Bresenham's Circle Algorithm for calculation of the locations of the pixels in the first octant of 45 degrees. It assumes that the circle is centered on the origin. So for every pixel (x, y) it calculates, we draw a pixel in each of the 8 octants of the circle as shown below :



For a pixel (x, y) , all possible pixels in 8 octants

Now, we will see how to calculate the next pixel location from a previously known pixel location (x, y) . In Bresenham's algorithm at any point (x, y) we have two options either to choose the next pixel in the east i.e. $(x+1, y)$ or in the south east i.e. $(x+1, y-1)$.



And this can be decided by using the decision parameter d as:

- If $d > 0$, then $(x+1, y-1)$ is to be chosen as the next pixel as it will be closer to the arc.
- else $(x+1, y)$ is to be chosen as next pixel.

Now to draw the circle for a given radius ' r ' and centre (x_c, y_c) We will start from $(0, r)$ and move in first quadrant till $x=y$ (i.e. 45 degree). We should start from listed initial condition:

$$d = 3 - (2 * r)$$

$$x = 0$$

$$y = r$$

Now for each pixel, we will do the following operations:

1. Set initial values of (x_c, y_c) and (x, y) .
2. Set the decision parameter d to $d = 3 - (2 * r)$.
3. Call the `drawCircle(int xc, int yc, int x, int y)` function.
4. Repeat the following steps until $x \leq y$:
 - If $d < 0$, set $d = d + (4 * x) + 6$.
 - Else, set $d = d + 4 * (x - y) + 10$ and decrement y by 1.
 - Increment the value of x .
 - Call the `drawCircle(int xc, int yc, int x, int y)` function.

drawCircle() function:

// function to draw all other 7 pixels

// present at symmetric position

`drawCircle(int xc, int yc, int x, int y)`

```

{
    putpixel(xc+x, yc+y, RED);
    putpixel(xc-x, yc+y, RED);
    putpixel(xc+x, yc-y, RED);
    putpixel(xc-x, yc-y, RED);
    putpixel(xc+y, yc+x, RED);
    putpixel(xc-y, yc+x, RED);
    putpixel(xc+y, yc-x, RED);
    putpixel(xc-y, yc-x, RED);
}

```

Below is C implementation of above approach.

```
// C-program for circle drawing
```

```
// using Bresenham's Algorithm
```

```
// in computer-graphics
```

```
#include <stdio.h>
```

```
#include <dos.h>
```

```
#include <graphics.h>
```

```
// Function to put pixels
```

```
// at subsequence points
```

```
void drawCircle(int xc, int yc, int x, int y){
```

```
    putpixel(xc+x, yc+y, RED);
```

```
    putpixel(xc-x, yc+y, RED);  
  
    putpixel(xc+x, yc-y, RED);  
  
    putpixel(xc-x, yc-y, RED);  
  
    putpixel(xc+y, yc+x, RED);  
  
    putpixel(xc-y, yc+x, RED);  
  
    putpixel(xc+y, yc-x, RED);  
  
    putpixel(xc-y, yc-x, RED);  
}
```

```
// Function for circle-generation
```

```
// using Bresenham's algorithm
```

```
void circleBres(int xc, int yc, int r){
```

```
    int x = 0, y = r;
```

```
    int d = 3 - 2 * r;
```

```
    drawCircle(xc, yc, x, y);
```

```
    while (y >= x){
```

```
        // check for decision parameter
```

```
        // and correspondingly
```

```
        // update d, y
```

```
        if (d > 0) {
```

```
            y--;
```

```

        d = d + 4 * (x - y) + 10;
    }

    else

        d = d + 4 * x + 6;

    // Increment x after updating decision parameter

    x++;

    // Draw the circle using the new coordinates

    drawCircle(xc, yc, x, y);

    delay(50);
}
}

int main()
{
    int xc = 50, yc = 50, r = 30;

    int gd = DETECT, gm;

    initgraph(&gd, &gm, ""); // initialize graph

    circleBres(xc, yc, r); // function call

    return 0;
}

```

Output:



Advantages

- It is a simple algorithm.
- It can be implemented easily
- It is totally based on the equation of circle i.e. $x^2 + y^2 = r^2$

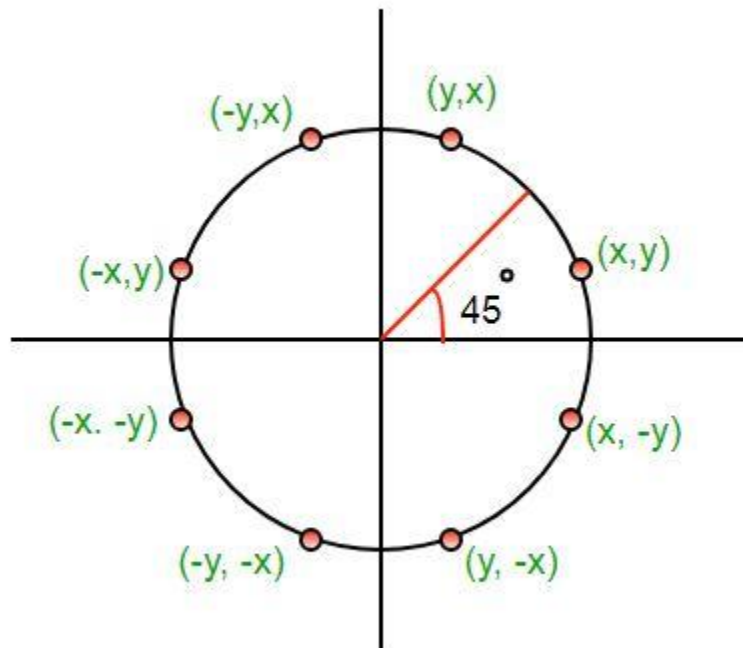
Disadvantages

- There is a problem of accuracy while generating points.
- This algorithm is not suitable for complex and high graphic images.

Mid-Point Circle Drawing Algorithm

The **mid-point** circle drawing algorithm is an algorithm used to determine the points needed for rasterizing a circle.

We use the **mid-point** algorithm to calculate all the perimeter points of the circle in the **first octant** and then print them along with their mirror points in the other octants. This will work because a circle is symmetric about its centre.



The algorithm is very similar to the [Mid-Point Line Generation Algorithm](#). Here, only the boundary condition is different.

For any given pixel (x, y) , the next pixel to be plotted is either $(x, y+1)$ or $(x-1, y+1)$. This can be decided by following the steps below.

1. Find the mid-point p of the two possible pixels i.e $(x-0.5, y+1)$
2. If p lies inside or on the circle perimeter, we plot the pixel $(x, y+1)$, otherwise if it's outside we plot the pixel $(x-1, y+1)$

Boundary Condition : Whether the mid-point lies inside or outside the circle can be decided by using the formula:-

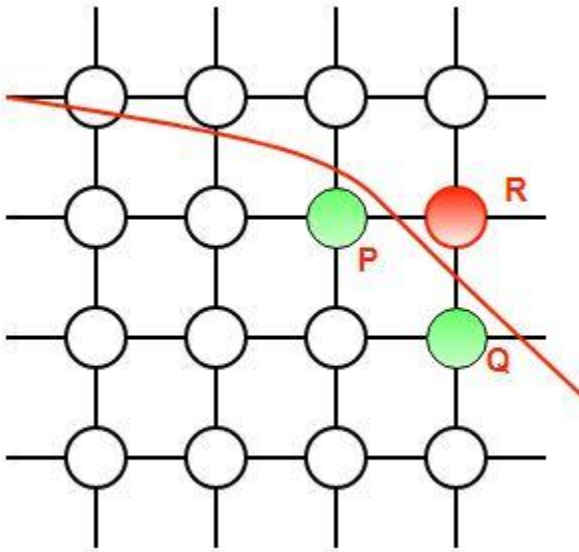
Given a circle centered at (0,0) and radius r and a point $p(x,y)$

$$F(p) = x^2 + y^2 - r^2$$

if $F(p) < 0$, the point is inside the circle

$F(p) = 0$, the point is on the perimeter

$F(p) > 0$, the point is outside the circle



In our program, we denote $F(p)$ with P . The value of P is calculated at the mid-point of the two contending pixels i.e. $(x-0.5, y+1)$. Each pixel is described with a subscript k .

$$P_k = (X_k - 0.5)^2 + (y_k + 1)^2 - r^2$$

Now,

$$x_{k+1} = x_k \text{ or } x_{k-1}, y_{k+1} = y_k + 1$$

$$? P_{k+1} = (x_{k+1} - 0.5)^2 + (y_{k+1} + 1)^2 - r^2$$

$$= (x_{k+1} - 0.5)^2 + [(y_k + 1) + 1]^2 - r^2$$

$$= (x_{k+1} - 0.5)^2 + (y_k + 1)^2 + 2(y_k + 1) + 1 - r^2$$

$$= (x_{k+1} - 0.5)^2 + [-(x_k - 0.5)^2 + (x_k - 0.5)^2] + (y_k + 1)^2 - r^2 + 2(y_k + 1) + 1$$

$$= P_k + (x_{k+1} - 0.5)^2 - (x_k - 0.5)^2 + 2(y_k + 1) + 1$$

$$= P_k + (x_{k+1}^2 - x_k^2) - (x_{k+1} - x_k) + 2(y_k + 1) + 1$$

$$= P_k + 2(y_k + 1) + 1, \text{ when } P_k \leq 0 \text{ i.e. the midpoint is inside the circle}$$

$$(x_{k+1} = x_k)$$

$P_k + 2(y_k + 1) - 2(x_k - 1) + 1$, when $P_k > 0$ i.e the mid point is outside the circle ($x_{k+1} = x_k - 1$)

The first point to be plotted is (r, 0) on the x-axis. The initial value of P is calculated as follows:-

$$\begin{aligned} P_1 &= (r - 0.5)^2 + (0+1)^2 - r^2 \\ &= 1.25 - r \\ &= 1 - r \text{ (When rounded off)} \end{aligned}$$

Examples:

Input : Centre -> (0, 0), Radius -> 3

Output : (3, 0) (3, 0) (0, 3) (0, 3)

(3, 1) (-3, 1) (3, -1) (-3, -1)

(1, 3) (-1, 3) (1, -3) (-1, -3)

(2, 2) (-2, 2) (2, -2) (-2, -2)

(x1,y1) is initially printed before the loop: (3,0) (3,0) (0,3) (0,3)

k	P _k	X _k	Y _k	P _{k+1}	X _{k+1}	Y _{k+1}	Output
1	-2	3	0	-1	3	1	(3,1) (-3,1) (3,-1) (-3,-1) (1,3) (-1,3) (1,-3) (-1,-3)
2	-1	3	1	2	2	2	(2,2) (-2,2) (2,-2) (-2,-2)
3	2	2	2				Break from loop

Input : Centre -> (4, 4), Radius -> 2

Output : (6, 4) (6, 4) (4, 6) (4, 6)

(6, 5) (2, 5) (6, 3) (2, 3)

(5, 6) (3, 6) (5, 2) (3, 2)

2.3 Polygons – Types of polygons, Inside Outside test, polygon filling: Seed fill algorithms: flood fill, boundry fill, scan-line algorithms.

A polygon is a closed [two-dimensional figure](#) composed of straight-line segments that meet at their endpoints. The line segments of polygons are called sides, and each endpoint is called a vertex. Polygons have at least three sides and three angles, and their sides must be straight, as seen below.

A polygon cannot have any curve sides. Open-sided figures and shapes with curves are examples of non-polygon figures, as seen below.

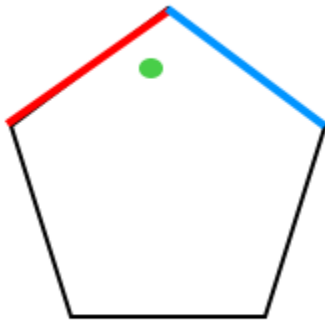
Polygon is an ordered list of vertices as shown in the following figure. For filling polygons with particular colors, you need to determine the pixels falling on the border of the polygon and those which fall inside the polygon. In this chapter, we will see how we can fill polygons using different techniques.



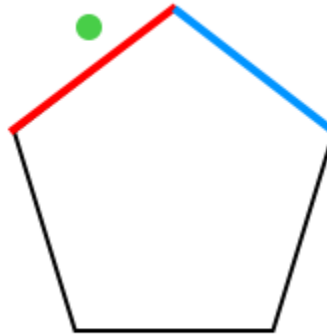
How to check if a given point lies inside or outside a polygon?

Given a **polygon** and a point '**p**', find if '**p**' lies inside the polygon or not. The points lying on the border are considered inside.

Examples:



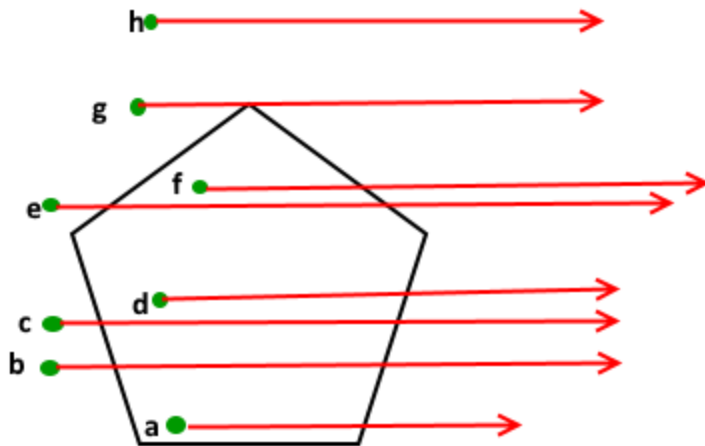
Inside example



outside example

Approach: The idea to solve this problem is based on [How to check if two given line segments intersect](#), and to be used as follows:

1. Draw a horizontal line to the right of each point and extend it to infinity
2. Count the number of times the line intersects with polygon edges.
3. A point is inside the polygon if either count of intersections is odd or point lies on an edge of polygon. If none of the conditions is true, then point lies outside.



How to handle point 'g' in the above figure?

Note that we should return true if the point lies on the line or the same as one of the vertices of the given polygon. To handle this, after checking if the line from 'p' to extreme intersects, we check whether 'p' is collinear with vertices of the current line of polygon. If it is collinear, then we check if the point 'p' lies on current side of polygon, if it lies, we return true, else false.

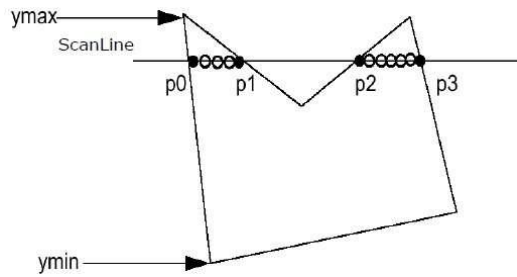
Seed fill algorithms

Seed fill algorithms are used to fill a connected region in an image, starting from a given seed point. These algorithms are commonly used in paint programs (like the "bucket fill" tool) and graphics rendering.

Scan Line Algorithm

This algorithm works by intersecting scanline with polygon edges and fills the polygon between pairs of intersections. The following steps depict how this algorithm works.

Step 1 – Find out the Ymin and Ymax from the given polygon.



Step 2 – ScanLine intersects with each edge of the polygon from Ymin to Ymax. Name each intersection point of the polygon. As per the figure shown above, they are named as p0, p1, p2, p3.

Step 3 – Sort the intersection point in the increasing order of X coordinate i.e. (p0, p1), (p1, p2), and (p2, p3).

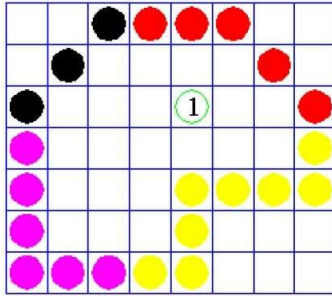
Step 4 – Fill all those pair of coordinates that are inside polygons and ignore the alternate pairs.

Flood Fill Algorithm

Sometimes we come across an object where we want to fill the area and its boundary with different colors. We can paint such objects with a specified interior color instead of searching for particular boundary color as in boundary filling algorithm.

Instead of relying on the boundary of the object, it relies on the fill color. In other words, it replaces the interior color of the object with the fill color. When no more pixels of the original interior color exist, the algorithm is completed.

Once again, this algorithm relies on the Four-connect or Eight-connect method of filling in the pixels. But instead of looking for the boundary color, it is looking for all adjacent pixels that are a part of the interior.



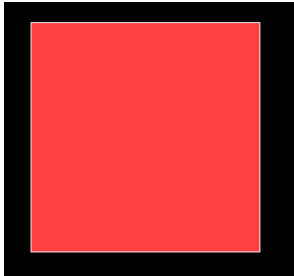
Given a rectangle, your task to fill this rectangle using flood fill algorithm.

Examples:

Input : rectangle(left = 50, top = 50, right= 100, bottom = 100)

flood(x = 55, y = 55, new_color = 12, old_color = 0)

Output :



Input : rectangle(left = 50, top = 50, right= 200, bottom = 400)

flood(x = 51, y = 51, new_color = 6, old_color = 0)

Output :



Flood fill algorithm fills new color until the old color match.

Flood fill algorithm:-

```
// A recursive function to replace previous
```

```
// color 'oldcolor' at '(x, y)' and all
```

```
// surrounding pixels of (x, y) with new
```

```
// color 'newcolor' and
```

```
floodfill(x, y, newcolor, oldcolor)
```

1) If x or y is outside the screen, then

return.

2) If color of getpixel(x, y) is same as

oldcolor, then

3) Recur for top, bottom, right and left.

```
floodFill(x+1, y, newcolor, oldcolor);
```

```
floodFill(x-1, y, newcolor, oldcolor);
```

```
floodFill(x, y+1, newcolor, oldcolor);
```

```
floodFill(x, y-1, newcolor, oldcolor);
```

Boundary Fill Algorithm

The boundary fill algorithm works as its name. This algorithm picks a point inside an object and starts to fill until it hits the boundary of the object. The color of the boundary and the color that we fill should be different for this algorithm to work.

Introduction : Boundary Fill Algorithm starts at a pixel inside the polygon to be filled and paints the interior proceeding outwards towards the boundary. This algorithm works **only if** the color with which the region has to be filled and the color of the boundary of the region are different. If the boundary is of one single color, this approach proceeds outwards pixel by pixel until it hits the boundary of the region.

Boundary Fill Algorithm is recursive in nature. It takes an interior point(x, y), a fill color, and a boundary color as the input. The algorithm starts by checking the color of (x, y). If it's color is not equal to the fill color and the boundary color, then it is painted with the fill color and the function is called for all the neighbours of (x, y). If a point is found to be of fill color or of boundary color, the function does not call its neighbours and returns. This process continues until all points up to the boundary color for the region have been tested.

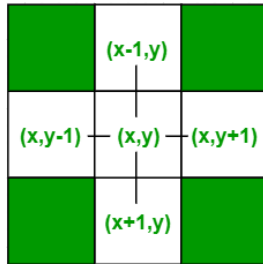
The boundary fill algorithm can be implemented by 4-connected pixels or 8-connected pixels.

4-connected pixels : After painting a pixel, the function is called for four neighbouring points. These are the pixel positions that are right, left, above, and below the current pixel. Areas filled by this method are called 4-connected. Below given is the algorithm :

Algorithm :

```
void boundaryFill4(int x, int y, int fill_color,int boundary_color)
```

```
{  
  
    if(getpixel(x, y) != boundary_color &&  
        getpixel(x, y) != fill_color)  
    {  
        putpixel(x, y, fill_color);  
  
        boundaryFill4(x + 1, y, fill_color, boundary_color);  
  
        boundaryFill4(x, y + 1, fill_color, boundary_color);  
  
        boundaryFill4(x - 1, y, fill_color, boundary_color);  
  
        boundaryFill4(x, y - 1, fill_color, boundary_color);  
    }  
}
```



Below is the implementation of above algorithm :

```
// C Implementation for Boundary Filling Algorithm

#include <graphics.h>

// Function for 4 connected Pixels

void boundaryFill4(int x, int y, int fill_color,int boundary_color)
{
    if(getpixel(x, y) != boundary_color &&
        getpixel(x, y) != fill_color)
    {
        putpixel(x, y, fill_color);

        boundaryFill4(x + 1, y, fill_color, boundary_color);

        boundaryFill4(x, y + 1, fill_color, boundary_color);

        boundaryFill4(x - 1, y, fill_color, boundary_color);

        boundaryFill4(x, y - 1, fill_color, boundary_color);
    }
}
```

```
    }  
}  
  
//driver code  
  
int main()  
{  
    // gm is Graphics mode which is  
    // a computer display mode that  
    // generates image using pixels.  
    // DETECT is a macro defined in  
    // "graphics.h" header file  
  
    int gd = DETECT, gm;  
  
    // initgraph initializes the  
    // graphics system by loading a  
    // graphics driver from disk  
    initgraph(&gd, &gm, "");  
  
    int x = 250, y = 200, radius = 50;  
  
    // circle function
```

```
circle(x, y, radius);

// Function calling
boundaryFill4(x, y, 6, 15);

delay(10000);

getch();

// closegraph function closes the
// graphics mode and deallocates
// all memory allocated by
// graphics system .
closegraph();

return 0;

}
```

Output :